ZentrFi — Technical Updates Report

Project: ZentrFi (zentrifi.xyz) — Web3 AI Travel Platform **Date:** March 8, 2026 **Stack:** Next.js 16 / React 19 / TypeScript / Tailwind CSS v4 **Prepared by:** Engineering Team

---

Overview

This document outlines six major technical updates implemented on the ZentrFi platform. Each section explains what was built, why it matters, and how it works — written so both developers and non-technical stakeholders can follow along.

Feature	Status	Description
XMTP Messaging	Built, flag off	Wallet-to-wallet encrypted chat
ENS / Basename Resolution	Built, flag off	Human-readable wallet names
Farcaster Frames v2	Built, flag off	Social media embed cards
MongoDB Trip History	Built, flag off	Persistent booking records
Telegram Bot	Live	Flight search via Telegram
Multi-Chain Payments	In progress	Pay from any blockchain

---

1. XMTP Messaging — Wallet-to-Wallet Encrypted Chat

What It Is

XMTP (Extensible Message Transport Protocol) is a decentralized messaging protocol that lets any two crypto wallet addresses communicate directly — no email, phone number, or account required. Think of it as "iMessage, but for wallets."

What We Built

We integrated the XMTP browser SDK into the ZentrFi dashboard, allowing users to:

- **Send and receive encrypted messages** using only their connected wallet address
- **Share travel itineraries** directly with other travelers or travel agents
- **Chat in real-time** through a dedicated messaging panel in the dashboard sidebar

How It Works (Technical)

1. When a user connects their wallet (via RainbowKit/WalletConnect), the app checks if they have an XMTP identity
2. If not, the user signs a one-time message to create their XMTP keypair
3. Messages are end-to-end encrypted and stored on the XMTP network — ZentrFi never sees message contents
4. The chat panel uses XMTP's browser SDK to stream incoming messages in real-time

Why It Matters

Traditional travel platforms force users to create accounts and share personal information. With XMTP, ZentrFi users can coordinate travel plans using nothing but their wallet address — fully private, fully decentralized.

> **Current Status:** Feature is fully built and functional. Currently disabled via feature flag (`NEXT_PUBLIC_FF_XMTP=false`) while we finalize UI polish and testing.

---

2. ENS / Basename Resolution — Human-Readable Wallet Names

What It Is

Ethereum Name Service (ENS) turns long, hard-to-read wallet addresses like `0x705e...d51C` into simple, memorable names like `alice.eth`. Basename is the equivalent system on the Base blockchain. Together, they make the platform feel more personal and user-friendly.

What We Built

Wherever a wallet address appears in the ZentrFi dashboard, the system now:

- **Automatically looks up** the ENS name (on Ethereum) or Basename (on Base)
- **Displays the human-readable name** instead of the raw address (e.g., "alice.eth" instead of "0x705e...d51C")
- **Falls back gracefully** to a shortened address format if no name is registered

How It Works (Technical)

1. A custom React hook (`useResolvedName`) is triggered when a wallet connects
2. The hook queries the ENS and Base name registries on-chain using the connected wallet address
3. Results are cached so we don't re-query on every page load

4. The resolved name is displayed in the header, chat messages, and trip history

Why It Matters

Wallet addresses are intimidating for new users. Showing "alice.eth" instead of "0x705e...d51C" makes the platform feel more human, builds trust, and helps users verify they're interacting with the right person.

> **Current Status:** Fully implemented. Disabled via feature flag (`NEXT_PUBLIC_FF_ENS_RESOLVE=false`) pending final QA testing across multiple wallet providers.

---

3. Farcaster Frames v2 — Social Media Embed Cards

What It Is

Farcaster is a decentralized social media platform (similar to X/Twitter). Frames v2 is Farcaster's system for embedding interactive cards directly inside social media posts — similar to how a YouTube link shows a video preview on Twitter.

What We Built

We added Farcaster Frames v2 meta tags to the ZentrFi application, which means:

- **When someone shares a ZentrFi link on Farcaster**, it automatically renders as a rich interactive card
- The card shows the **ZentrFi branding, description, and a call-to-action** button

- Users can **interact with ZentrFi directly from their Farcaster feed** without leaving the app

How It Works (Technical)

1. Special `fc:frame` meta tags were added to the application's HTML `<head>` section
2. These tags define the frame's image, title, description, and action buttons
3. Farcaster Neynar API integration (with webhook support) enables the platform to respond to frame interactions
4. The implementation includes a bot signer UUID for authenticated Farcaster actions

Why It Matters

Farcaster is one of the fastest-growing Web3 social platforms. Frames v2 lets ZentrFi meet users where they already are — if someone shares a flight deal on Farcaster, their followers can see it as an interactive card and start searching flights immediately.

> **Current Status:** Meta tags and Neynar integration are fully configured. Disabled via feature flag (`NEXT_PUBLIC_FF_FRAMES_V2=false`) while we await Farcaster's Frames v2 mainnet rollout.

---

4. MongoDB Trip History — Persistent Booking Records

What It Is

Every flight booking made through ZentrFi is now permanently stored in a MongoDB database, linked to the user's wallet address. This means users can come back anytime and see their complete travel history — no account or email required, just their wallet.

What We Built

- A **MongoDB connection layer** that securely connects the ZentrFi backend to a cloud-hosted MongoDB Atlas cluster
- **Automatic booking storage**: when a user completes a flight booking, all details are saved — airline, route, price, stops, duration, passenger name, confirmation ID, and timestamp
- A **Trip History sidebar** on the dashboard that displays all past bookings as styled cards with flight details and a "Confirmed" status badge
- **Wallet-scoped queries**: each user only sees their own bookings, identified by their connected wallet address

How It Works (Technical)

1. A singleton MongoDB client (`src/server/mongodb.ts`) maintains a persistent connection to MongoDB Atlas
2. When a booking completes via the Duffel API, the `/api/book-flight` endpoint inserts a record into the `bookings` collection
3. The Trip History sidebar calls `/api/trip-history?wallet=<address>`, which queries MongoDB for all bookings matching that wallet
4. Results are sorted by date (newest first), limited to 50 entries, and displayed as enriched cards showing airline, route, price, duration, and booking date

Data Stored Per Booking

Field	Example

Confirmation ID	`ord_0000B43ZRgYFy9wWAlnwvY`
Wallet Address	`0x705e...d51c`
Airline	British Airways
Route	ATL → MIA
Price	$83.68 USD
Stops	Direct
Duration	1h 51m
Passenger Name	John Smith
Status	Confirmed

Why It Matters

On traditional travel sites, your booking history is locked behind an email/password account. On ZentrFi, your wallet IS your account. Connect your wallet, and your complete travel history is right there — no passwords, no account recovery headaches.

> **Current Status:** Backend is fully operational and tested. MongoDB connection confirmed. Disabled via feature flag (`NEXT_PUBLIC_FF_TRIP_HISTORY=false`) pending end-to-end booking flow testing on staging.

---

5. Telegram Bot (@zentrfi_bot) — Flight Search via Telegram

What It Is

A Telegram bot that connects directly to the ZentrFi backend, allowing users to search for real-time flights without ever opening a web browser. Users send a message like "Flights from Atlanta to Miami" and receive actual prices from airlines — the same results they'd see on the ZentrFi dashboard.

What We Built

- A **standalone Node.js bot** (`scripts/telegram-bot.mjs`) using the Grammy framework
- **Natural language trip parsing**: the bot sends user messages to ZentrFi's `/api/plan-trip` endpoint, which uses Google Gemini AI to extract the origin, destination, and travel date
- **Airport resolution**: city names are automatically converted to IATA airport codes using the `/api/flights/places` endpoint. If multiple airports match (e.g., "New York" → JFK, LGA, EWR), the bot shows **inline keyboard buttons** for the user to tap and select
- **Real-time flight search**: once airports are resolved, the bot calls `/api/flights/search` — the exact same Duffel API endpoint that powers the web dashboard
- **Formatted results** showing airline name, price in USD, number of stops, and flight duration

How It Works (Technical)

1. User sends a message to `@zentrfi_bot` on Telegram
2. Bot forwards the message to `/api/plan-trip` → Gemini AI parses origin, destination, date
3. Bot resolves city names to IATA codes via `/api/flights/places` (with button fallback for ambiguous cities)

4. Bot calls `/api/flights/search` with the IATA codes → Duffel returns real-time offers
5. Bot formats the top 6 results and sends them back with a "Book with crypto" link to zentrifi.xyz

Example Conversation

> **User:** I want to make a trip from ATL GA to Miami FL **Bot:** Searching ATL GA → Miami FL on 2026-03-15... **Bot:** ATL → MIA · 2026-03-15
>
> 1. British Airways — $83.68 (Direct · 1h 51m)
> 2. Iberia — $83.74 (Direct · 1h 51m)
> 3. American Airlines — $85.02 (Direct · 1h 51m) Book with crypto: zentrifi.xyz

Why It Matters

Not everyone wants to open a website to check flight prices. With the Telegram bot, users can search flights from any device, in any chat — and get the same real-time prices powered by the Duffel API. It extends ZentrFi's reach beyond the browser.

> **Current Status:** Live and operational. Currently connected to the development server. Production deployment requires updating the API URL to the Vercel-hosted backend.

---

6. Multi-Chain Payments — Pay From Any Blockchain (In Progress)

What It Is

Multi-Chain Payments will allow users to pay for flights using tokens from **any supported blockchain** — not just Ethereum or Base. Using the LI.FI SDK (a cross-chain bridging protocol), the system will automatically bridge funds from the user's chain to the payment chain in a single transaction.

What's Planned

- **Cross-chain token bridging** via LI.FI SDK: if a user has USDC on Polygon but the payment requires USDC on Base, the system handles the bridge automatically
- **Multi-wallet support**: pay from whichever chain has available funds
- **Transparent fee display**: users see the bridge fee and total cost before confirming
- **Fallback to native payments**: if bridging isn't needed (user is already on the right chain), the payment goes through directly

How It Will Work (Technical)

1. User selects a flight and enters booking details
2. The system detects which chain the user's wallet is connected to
3. If a cross-chain bridge is needed, LI.FI SDK calculates the optimal route and fee
4. User approves a single transaction that handles bridging + payment
5. Booking completes once payment is confirmed on-chain

Why It Matters

Today, most Web3 apps only accept payments on one or two chains. This forces users to manually bridge tokens before they can pay — a confusing and error-prone

process. Multi-Chain Payments will remove this friction entirely, making ZentrFi accessible to users on any chain.

> **Current Status:** This feature is currently **on hold**. The UI component is built but greyed out in the dashboard (feature flag `NEXT_PUBLIC_FF_MULTICHAIN_PAY=false`). Development will resume once the core booking flow is fully stabilized. LI.FI SDK integration is scoped and ready for implementation.

Architecture Summary

```
User (Browser)                    User (Telegram)
     |                                  |
     ▼                                  ▼
┌──────────────┐              ┌──────────────────┐
│  Next.js     │              │  telegram-bot.mjs │
│  Dashboard   │              │  (Grammy + Node)  │
└──────┬───────┘              └──────────┬───────┘
       |                                  |
       ▼                                  ▼
┌──────────────────────────────────────────┐
│          ZentrFi API Layer               │
│  /api/plan-trip    (Gemini AI parsing)   │
│  /api/flights/*    (Duffel real-time)    │
│  /api/book-flight  (Duffel + MongoDB)    │
│  /api/trip-history (MongoDB query)       │
└──────────────────┬───────────────────────┘
                   |
          ┌────────┴────────┐
          ▼                 ▼
    ┌──────────┐      ┌──────────┐
```

```
    │ Duffel    │      │ MongoDB    │
    │ (Flights)│      │ (Bookings)│
    └──────────┘      └────────────┘
```

This document is confidential and intended for internal use by the ZentrFi development team.